Project: Extraction of aspect-based opinions from customer reviews

NLP Course @CentraleSupélec - 2025-2026 - S. Aït-Mokhtar

The deadline for submitting the project deliverable is April 30, 2026.
No oral presentation.
You can work on the project in groups of 4 people (max).

Table of contents

- 1. Introduction
- 2. How to proceed
  - 2.1. Installing Python and libraries
  - 2.2. Downloading the assignment file
    - 2.2.1. The data
    - 2.2.2. Code base and your expected contribution
  - 2.3. Possible approaches
    - 2.3.1. Approach 1: No fine-tuned LLM
    - 2.3.2. Approach 2: LoRA fine-tuned LLM
    - 2.3.3. Approach 3: Fine-tuned encoder-only LM
- 3. Expected project deliverable
- 4. Evaluation of the assignment

1. Introduction

The project topic is the extraction of aspect-based opinions from customer reviews about restaurants. The objective is to implement a tool that extracts the overall opinions on 3 predefined aspects:

1. Price: price of dishes and drinks
2. Food: quality and quantity of food and drinks offered.
3. Service: quality and efficiency of the service and reception.

For each review and each aspect, the tool must predict one of the 4 possible values (classes) for the overall opinion on the aspect, namely:

1. Positive: when the review contains one or more positive opinions on the given aspect, and no negative opinions on this aspect.
2. Negative: when the review contains only one or more negative opinions on the given aspect and no positive opinions on this aspect.
3. Mixed: when the review expresses at least one positive and at least one negative opinion on the given aspect.
4. No Opinion: when the review does not contain any expressed opinion on the given aspect.

You can follow one of these 3 approaches (see list of authorized models in the corresponding sections below)

1. No fine-tuning: LLM-based zero-shot or few-shot in-context-learning
2. LLM fine-tuning, with LoRA.
3. Masked LM fine-tuning (i.e. encoder-only pretrained LMs)

2. How to proceed

Install the specified Python and libraries, then choose an approach and follow the corresponding guidelines below.

2.1. Installing Python and libraries

1. Create a new Python environment dedicated to this assignment, with `python 3.12`
2. Activate the newly created environment
3. Install the following libraries, with the specified versions:
   1. pytorch 2.9.1 (follow the instructions here: https://pytorch.org/get-started/previous-versions/)
   2. transformers 5.5.0
   3. datasets 4.8.4
   4. openai 2.31.0

5. langchain 1.2.15
6. langchain-openai 1.1.12
7. sentencepiece 0.2.1
8. trl 1.0.0
9. accelerate 1.13.0
10. peft 0.18.1
11. pandas 3.0.2
12. pyrallis 0.3.1

IMPORTANT: Only these listed libraries (with the specified versions), plus the standard libraries that come with Python 3.12.x, are allowed for this assignment.

2.2. Downloading the assignment file

Download the `nlpproject.zip` the compressed file.
After extraction, you have a root directory `nlpproject` that contains:

1. An `src` subdirectory containing code base from which you start.
2. A `data` subdirectory containing the training, development (or validation) in `tsv` format. Please do not displace or rename these files; they are used by the code in `src`.

The first thing to do is rename the root directory so that it is formed from the name or names of the authors of the deliverable, so that when you compress it in ZIP format, the resulting compressed file has a name compatible with the guidelines below.

You should not modify the structure inside this root directory.

2.2.1. The data

Each row in the data contains 5 fields: name of the restaurant, opinion values for "Price", "Food" and "Service", and the customer review, in french.

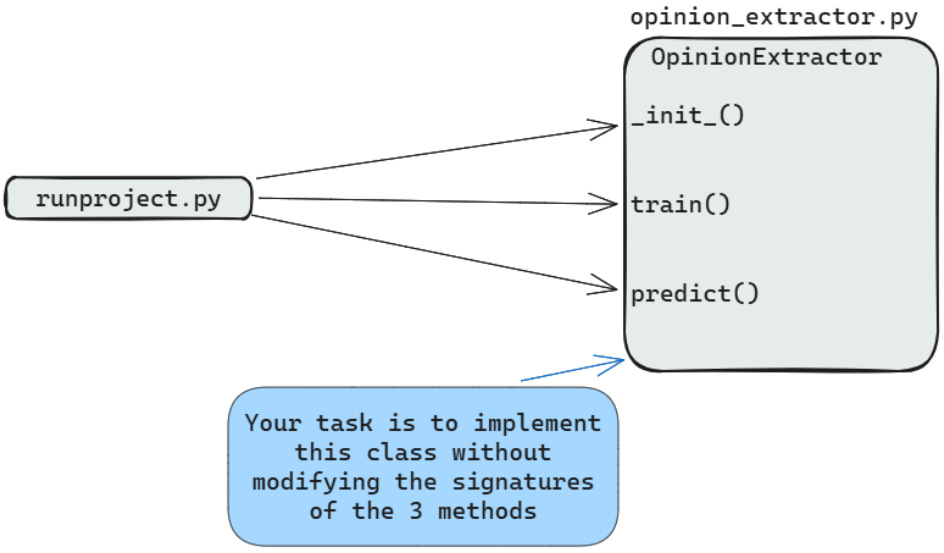
There are two files: the train set and the validation/dev set. Their annotations are a bit noisy. There is a manually curated test set (not released) on which your program will be evaluated.

2.2.2. Code base and your expected contribution

The `src` folders comes with 2 source files that **should not be** modified:

- `config.py`
- `runproject.py`

There is a third python file, `opinion_extractor.py` containing a class named `OpinionExtractor` which has 3 methods: `__init__()`, `train() and` predict() `. These methods are called from the` runproject.py` script to: (1) Initialize the opinion extractor instance; (2) to train its model (if method is set to "FT"), and (3) run predictions in order to evaluate the extractor on the dev or test data.



Your task is to implement these 3 methods without modifying their signature. The implementation of these methods depends on which approach you choose for building the opinion extraction system (see the section about the 3 possible approaches below). You need to

set the class field "method" of `OpinionExtractor` to "FT" if you choose an approach that require fine-tuning, and "NOFT" otherwise.

Your program must work without modifying the `config.py` and `runproject.py`. If it does not, then there is something incorrect in your implementation and it should be fixed within the `opinion_extractor.py` file.

Again, please check that everything works by going to the `src` folder, and running the following command:

```
accelerate launch runproject.py
```

2.3. Possible approaches

Depending on which approach you choose, you will need to set the class field "method" of `OpinionExtractor` to "FT" (fine-tuning) or "NOFT" (no fine-tuning).

2.3.1. Approach 1: No fine-tuned LLM

For this approach, you need to set up an Ollama inference endpoint and then use either the `openai` or `langchain` python library to send prompts to the endpoint and get responses.

First install the Ollama server: follow the instructions here: https://ollama.com/download

Then select a model from the list of authorized models below and download it with the ollama command line, for instance:

```
ollama pull gemma3:4b
```

For this approach, the list of authorized LLMs (with their exact ollama identifiers) is the following:

gemma3:4b
qwen3:4b
qwen3:4b-instruct
qwen3.5:4b
nemotron-3-nano:4b
gemma4:e4b
qwen3:8b
ministral-3:8b
qwen3.5:9b

IMPORTANT: Please do not modify the default values in the provided `config.py` file. When you create the client of the LLM inference endpoint, you will need to specify `cfg.ollama_url` as the value of the `base_url` argument. If your Ollama sever has a different URL, then specify it in the command line with the `--ollama_url` option.

For instance, when using `langchain`, you can create the client like this:

```
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(
    api_key="",
    base_url=cfg.ollama_url,
    model="[cfg.ollama_model](gemma3:4b)",
)
```

If your Ollama server URL is different from the default one ('http://localhost:11434/v1'), then you can specify it from the command line, e.g.:

```
accelerate launch runproject.py --ollama_url=http://localhost:8100/v1
```

2.3.2. Approach 2: LoRA fine-tuned LLM

This approach consists in fine-tuning an LLM with LoRA, leveraging the Huggingface `TRL` and `PEFT` libraries. Full-parameter fine-tuning is not authorized. Have a look at the the python notebook about LoRA fine-tuning, provided with the "LLM Post-Training" lecture.

Assume the GPU VRAMs are limited to 32GB max, which will limit the possible values of the `per_device_train_batch_size` hyperparameter, depending on the size of the LLM being fine-tuned.

For this approach, the list of authorized LLMs is the following (with their exact Huggingface identifiers):

LLM	Possible values of per_device_train_batch_size
Qwen/Qwen3-0.6B	1 to 4
Qwen/Qwen3-1.7B	1 or 2
Qwen/Qwen3-4B	1

IMPORTANT: Please assume the training process can run on any number of GPUs, depending on which server is available. Therefore, load the base model with the `device_map` argument set to `None` so that `accelerate` can set up data parallelism (DDP) based on the available devices:

```
model = AutoModelForCausalLM.from_pretrained(
    self.base_model_id,
    dtype=torch.float16,
    device_map=None,
)
```

Also, compute the number of GPUs programmatically, and then set the **per-device-batch-size**, the **gradient accumulation** and the **learning rate** hyper-parameters accordingly, so that the **effective batch size** and the **learning rate** remain "adjusted", regardless of the number of devices being used:

```
# Get the number of devices
num_devices = max(1, torch.cuda.device_count())
```

In any case, check that your program runs correctly with:

```
accelerate launch runproject.py
```

2.3.3. Approach 3: Fine-tuned encoder-only LM

This approach consists in fine-tuning an encoder-only model (Masked LM) leveraging the Huggingface `transformers` library. Full-parameter fine-tuning is authorized. Have a look at the the python notebook about provided with lecture 2 (Transformer architecture and contextual embeddings): It contains an example of text classification with an MLM (bert). In this assignment, there 3 aspects to classify ("Price", "Food" and "Service"), so if you choose this approach you will have to implement a kind of 3-head classifier.

For this approach, the list of authorized encoder-only models, all from the Huggingface repository (with their exact identifiers):

flaubert/flaubert_small_cased
flaubert/flaubert_base_cased
flaubert/flaubert_base_uncased
flaubert/flaubert_large_cased
almanach/moderncamembert-base
FacebookAI/xlm-roberta-base
jhu-clsp/mmBERT-small
jhu-clsp/mmBERT-base

IMPORTANT: Please assume the training process can run on any number of GPUs (from 1 to 8), See the guidelines in Approach 2 above about how to load the models, how to adjust the per-device-batch_size, and how to launch with the `accelerate` command.

3. Expected project deliverable

Please upload your project deliverable (as a zip-compressed folder) to the assignment space of the NLP Course team on Microsoft Teams. It must meet the following conditions:

1. It must be in the form of a single compressed file in ZIP format (other formats are not accepted).
2. The name of the compressed file must consist of the family names of the authors, in the format: `Name1_Name2_Name3_Name4.zip` (assuming 4 authors)
3. The size of the compressed file must not exceed 2 MB (remove any model checkpoints, large logs or compiled code)

In addition, when the file is decompressed, the root directory (whose name is formed from the names of the authors, as explained above) must contain directly the following items, and only these items (for more details, see section 6):

Elément	Description
README.txt or README.md	A .txt or .md file that must contain: 1. The full names of the deliverable authors (max=4 authors) 2. A few paragraphs (1 to 3 pages max) describing your work. 3. The macro-average accuracy you obtain on the dev data split.
src	Subdirectory containing ALL the files required to run the project with the command: `accelerate launch run_project.py`

Therefore, please delete any other files or directories automatically generated by the development tools being used (model checkpoints, logs, PyCharm's .idea directory, etc.) before compressing the directory and uploading it.

⚠ Before submitting the deliverable, please check that by going into the `src` directory and typing the following command, the program runs successfully:

```
accelerate launch runproject.py
```

4. Evaluation of the assignment

The grade will be calculated based on the following criteria:

- Macro-average accuracy across the 3 aspects (out of 12 points): macro-average of the 3 accuracy measurements (across the 3 aspects). Accuracy for an aspect is the percentage of opinions for which the predicted opinion values for that aspect are correct.
- Computational efficiency (4 points): execution time of the training process (if a model is trained) plus the execution time of the inference process (prediction of opinions on the test part).
- Quality of description of the delivered work in the README file (out of 4 points)

IMPORTANT: Penalty points will be applied to the final grade if:

- The project is submitted late (-1 point per day of delay)
- If the program does not work or produces an error during execution (minimum -5 points)
- The use of Python libraries that are not on the list of authorized libraries, or a language model (LM/LLM) that is not on the list of authorized LMs for the select approach (-10 points)
- If the submission format/structure requirements are not met (file name, format, structure, etc. see the sections below)